

CODIGO EN R DE COMPARACION:

```
# =====  
  
# COMPARACIÓN DE MÉTODOS CON 100 000 DATOS  
# MÍNIMOS CUADRADOS vs GRADIENTE DESCENDENTE  
# =====  
  
set.seed(123)  
  
# -----  
# 1. GENERACIÓN DE DATOS  
# -----  
  
n <- 100000  
x <- runif(n, 0, 100)      # Variable independiente  
y <- 12 + 3.7 * x + rnorm(n, 0, 20) # Relación lineal con ruido  
  
cat("Se generaron", n, "datos aleatoriamente.\n\n")  
  
# -----  
# 2. MÉTODO DE MÍNIMOS CUADRADOS  
# -----  
  
cat("==== MÉTODO: MÍNIMOS CUADRADOS =====\n")  
  
t_inicio_ls <- Sys.time()  
mem_inicio_ls <- sum(gc()[, 2])  
  
mean_x <- mean(x)  
mean_y <- mean(y)  
b_ls <- sum((x - mean_x) * (y - mean_y)) / sum((x - mean_x)^2)  
a_ls <- mean_y - b_ls * mean_x  
y_pred_ls <- a_ls + b_ls * x
```

```

mse_ls <- mean((y - y_pred_ls)^2)

mem_final_ls <- sum(gc()[, 2])
mem_usada_ls <- (mem_final_ls - mem_inicio_ls) / 1024 # MB
t_final_ls <- Sys.time()
tiempo_ls <- as.numeric(difftime(t_final_ls, t_inicio_ls, units = "secs"))

```

```

cat("a =", round(a_ls, 4), " | b =", round(b_ls, 4), "\n")
cat("MSE =", round(mse_ls, 6), "\n")
cat("Tiempo:", tiempo_ls, "segundos\n")
cat("Memoria usada:", round(mem_usada_ls, 6), "MB\n\n")

```

```

# -----
# 3. MÉTODO DEL GRADIENTE DESCENDENTE
# -----
cat("==== MÉTODO: GRADIENTE DESCENDENTE ==== \n")

```

```

# Normalización de los datos

```

```

x_norm <- scale(x)
y_norm <- scale(y)

```

```

a <- 0
b <- 0
alpha <- 0.01 # Tasa de aprendizaje
iteraciones <- 800 # Más iteraciones para convergencia estable
errores <- numeric(iteraciones)

```

```

t_inicio_gd <- Sys.time()
mem_inicio_gd <- sum(gc()[, 2])

```

```

for (i in 1:iteraciones) {

```

```

y_pred <- a + b * x_norm
da <- -2 * mean(y_norm - y_pred)
db <- -2 * mean((y_norm - y_pred) * x_norm)
a <- a - alpha * da
b <- b - alpha * db
errores[i] <- mean((y_norm - y_pred)^2)
}

mem_final_gd <- sum(gc()[, 2])
mem_usada_gd <- (mem_final_gd - mem_inicio_gd) / 1024 # MB
t_final_gd <- Sys.time()
tiempo_gd <- as.numeric(difftime(t_final_gd, t_inicio_gd, units = "secs"))

# Desnormalización
sx <- sd(x)
sy <- sd(y)
meanx <- mean(x)
meany <- mean(y)
b_gd <- b * (sy / sx)
a_gd <- meany + sy * a - b_gd * meanx
y_pred_gd <- a_gd + b_gd * x
mse_gd <- mean((y - y_pred_gd)^2)

cat("a =", round(a_gd, 4), " | b =", round(b_gd, 4), "\n")
cat("MSE =", round(mse_gd, 6), "\n")
cat("Tiempo:", tiempo_gd, "segundos\n")
cat("Memoria usada:", round(mem_usada_gd, 6), "MB\n\n")

# -----
# 4. COMPARACIÓN FINAL
# -----

```

```

cat("==== COMPARACIÓN FINAL ====\\n")

resultado <- data.frame(
  Metodo = c("Mínimos Cuadrados", "Gradiente Descendente"),
  a = c(a_ls, a_gd),
  b = c(b_ls, b_gd),
  MSE = c(mse_ls, mse_gd),
  Tiempo_seg = c(tiempo_ls, tiempo_gd),
  Memoria_MB = c(mem_usada_ls, mem_usada_gd)
)

print(resultado)

# -----
# 5. GRÁFICOS
# -----

par(mfrow = c(1, 2))

# Rectas de ajuste
plot(x[1:1000], y[1:1000], pch = 19, col = "lightblue",
     main = "Comparación de Ajustes (100 000 datos, muestra 1000)",
     xlab = "X", ylab = "Y")
abline(a_ls, b_ls, col = "red", lwd = 2)
abline(a_gd, b_gd, col = "green", lwd = 2, lty = 2)
legend("topleft", legend = c("Datos (1000)", "Mínimos Cuadrados", "Gradiente Desc."),
      col = c("lightblue", "red", "green"), pch = c(19, NA, NA), lty = c(NA, 1, 2))

# Convergencia del error
plot(1:iteraciones, errores, type = "l", col = "purple", lwd = 2,
     main = "Convergencia del Error (Gradiente)",
     xlab = "Iteraciones", ylab = "Error (MSE normalizado)")

```